# DESIGN AND ANALYSIS OF ALGORITHMS - UE24CS241B

## JACKFRUIT ASSIGNMENT
## QUESTION SET 13

SRN 1: PES1UG24CS493
NAME 1: Tanmayi Nagabhairava
SRN 2: PES1UG24CS500
NAME 2: Tejaswini R Pujar
SRN 3: PES1UG24CS514
NAME 3: Vaishnavi
SRN 4: PES1UG24CS516
NAME 4: Varsha A Patil
SEC: 4I, CSE DEPT

## CODES

### Question 1 – Divide and Conquer

You are given an unsorted array A[0…n−1] of n distinct integers. A rotation point is an index i where A[i] > A[i+1]. The array may have zero, one, or more rotation points. Design a divide-and-conquer algorithm to find the number of rotation points in the array.

**Solution:**

```
#include <stdio.h>

int countRotationLinear(int arr[], int n) {
   int count = 0;
   for (int i = 0; i < n - 1; i++) {
      if (arr[i] > arr[i + 1]) {
         count++;
      }
   }
   return count;
}

int countRotationDCUtil(int arr[], int low, int high) {
   if (low >= high) {
      return 0;
   }
   int mid = low + (high - low) / 2;

   int leftCount = countRotationDCUtil(arr, low, mid);
   int rightCount = countRotationDCUtil(arr, mid + 1, high);

   int crossCount = 0;
```

```
    if (mid < high && arr[mid] > arr[mid + 1]) {
        crossCount = 1;
    }
    return leftCount + rightCount + crossCount;
}

int countRotationDC(int arr[], int n) {
    if (n <= 1) {
        return 0;
    }
    return countRotationDCUtil(arr, 0, n - 1);
}

int main() {
    int arr1[] = {3, 8, 5, 7, 2};
    int arr2[] = {1, 2, 3, 4, 5};
    int arr3[] = {9, 6, 8, 4, 7, 2};
    int arr4[] = {10};
    int arr5[] = {-5, -2, -8, -1};

    int *tests[] = {arr1, arr2, arr3, arr4, arr5};
    int sizes[] = {5, 5, 6, 1, 4};

    for (int t = 0; t < 5; t++) {
        printf("Array: ");
        for (int i = 0; i < sizes[t]; i++) {
            printf("%d ", tests[t][i]);
        }
        printf("\n");

        printf("Linear Scan Rotation Points: %d\n", countRotationLinear(tests[t], sizes[t]));
        printf("Divide and Conquer Rotation Points: %d\n", countRotationDC(tests[t], sizes[t]));
        printf("--------------------------------------------------\n");
    }

    return 0;
}
```

## Question 2 – Greedy Method

You are given n coins with denominations $d_1$, $d_2$, …, $d_n$ (not necessarily the standard {1, 5, 10, 25}). Given an amount M, design a greedy algorithm to find the minimum number of coins needed to make the amount M. Assume an unlimited supply of each coin denomination.

**Solution:**

```
#include <stdio.h>

void sortDescending(int coins[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (coins[j] < coins[j + 1]) {
                int temp = coins[j];
                coins[j] = coins[j + 1];
```

```
            coins[j + 1] = temp;
        }
    }
  }
}

int removeInvalidAndDuplicateCoins(int coins[], int n) {
  int temp[1000];
  int newSize = 0;

  for (int i = 0; i < n; i++) {
    if (coins[i] <= 0) {
      continue;
    }

    int exists = 0;
    for (int j = 0; j < newSize; j++) {
      if (temp[j] == coins[i]) {
        exists = 1;
        break;
      }
    }

    if (!exists) {
      temp[newSize++] = coins[i];
    }
  }

  for (int i = 0; i < newSize; i++) {
    coins[i] = temp[i];
  }
  return newSize;
}

void greedyCoinChange(int coins[], int n, int amount) {
  if (amount < 0) {
    printf("Invalid amount\n");
    return;
  }

  if (amount == 0) {
    printf("Minimum coins by Greedy = 0\n");
    printf("No coins needed\n");
    return;
  }

  n = removeInvalidAndDuplicateCoins(coins, n);

  if (n == 0) {
    printf("No valid coin denominations available\n");
    return;
  }

  sortDescending(coins, n);

  int usedCount[1000] = {0};
  int totalCoins = 0;
```

```
    int originalAmount = amount;

    for (int i = 0; i < n; i++) {
        if (coins[i] <= amount) {
            usedCount[i] = amount / coins[i];
            totalCoins += usedCount[i];
            amount %= coins[i];
        }
    }

    printf("Coins: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", coins[i]);
    }
    printf("\n");

    printf("Amount: %d\n", originalAmount);

    if (amount != 0) {
        printf("Greedy could not form the exact amount\n");
    }
    else
    {
        printf("Minimum coins by Greedy = %d\n", totalCoins);
        printf("Coins used:\n");
        for (int i = 0; i < n; i++) {
            if (usedCount[i] > 0) {
                printf("%d coin(s) of %d\n", usedCount[i], coins[i]);
            }
        }
    }
    printf("-------------------------------------------------\n");
}

int main() {
    int coins1[] = {1, 5, 10, 25};
    int coins2[] = {1, 2, 5, 10};
    int coins3[] = {1, 3, 4};
    int coins4[] = {0, -2, 1, 1, 7, 10};
    int coins5[] = {5, 10};

    greedyCoinChange(coins1, 4, 63);
    greedyCoinChange(coins2, 4, 18);
    greedyCoinChange(coins3, 3, 6);
    greedyCoinChange(coins4, 6, 14);
    greedyCoinChange(coins5, 2, 3);

    printf("Failure Example:\n");
    printf("For coins {1, 3, 4} and amount 6\n");
    printf("Greedy gives 4 + 1 + 1 = 3 coins\n");
    printf("Optimal gives 3 + 3 = 2 coins\n");

    return 0;
}
```

## Question 3 – Dynamic Programming

You are given a set of n items, each with a weight $w_i$ and a value $v_i$, and a knapsack of capacity W. Design a dynamic programming algorithm to determine the maximum value that can be obtained by selecting items such that the total weight does not exceed W. Each item can be selected at most once (0/1 knapsack problem).

**Solution:**

```
#include <stdio.h>
#include <stdlib.h>

int max(int a, int b) {
   return (a > b) ? a : b;
}

void knapsack(int weights[], int values[], int n, int W) {
   int i, w;
   int **dp = (int **)malloc((n + 1) * sizeof(int *));
   for (i = 0; i <= n; i++) {
      dp[i] = (int *)malloc((W + 1) * sizeof(int));
   }

   for (i = 0; i <= n; i++) {
      for (w = 0; w <= W; w++) {
         if (i == 0 || w == 0) {
            dp[i][w] = 0;
         } else if (weights[i - 1] <= w) {
            dp[i][w] = max(dp[i - 1][w],
                     values[i - 1] + dp[i - 1][w - weights[i - 1]]);
         } else {
            dp[i][w] = dp[i - 1][w];
         }
      }
   }

   printf("Maximum Value = %d\n", dp[n][W]);
   printf("\nDP Table:\n");
   for (i = 0; i <= n; i++) {
      for (w = 0; w <= W; w++) {
         printf("%d ", dp[i][w]);
      }
      printf("\n");
   }

   int *selected = (int *)malloc(n * sizeof(int));
   int selectedCount = 0;
   w = W;
   for (i = n; i > 0 && w > 0; i--) {
      if (dp[i][w] != dp[i - 1][w]) {
         selected[selectedCount++] = i - 1;
         w -= weights[i - 1];
      }
   }
```

```
    printf("\nSelected items:\n");
    if (selectedCount == 0) {
        printf("No items selected\n");
    } else {
        for (i = selectedCount - 1; i >= 0; i--) {
            int idx = selected[i];
            printf("Item %d -> Weight = %d, Value = %d\n",
                   idx + 1, weights[idx], values[idx]);
        }
    }

    for (i = 0; i <= n; i++) {
        free(dp[i]);
    }
    free(dp);
    free(selected);
}

int main() {
    int weights1[] = {1, 3, 4, 5};
    int values1[] = {1, 4, 5, 7};
    int n1 = 4, W1 = 7;

    int weights2[] = {2, 3, 4, 5};
    int values2[] = {3, 4, 5, 6};
    int n2 = 4, W2 = 5;

    int weights3[] = {};
    int values3[] = {};
    int n3 = 0, W3 = 10;

    printf("Test Case 1\n");
    knapsack(weights1, values1, n1, W1);
    printf("-------------------------------------------------\n");

    printf("Test Case 2\n");
    knapsack(weights2, values2, n2, W2);
    printf("-------------------------------------------------\n");

    printf("Test Case 3\n");
    knapsack(weights3, values3, n3, W3);
    printf("-------------------------------------------------\n");

    return 0;
}
```